

1 Introduction to Digital Logic Design

Digital Logic Design (DLD) is the foundation of digital electronics and computer hardware. It involves designing circuits that process binary information (0s and 1s) using logic gates and other components. DLD bridges mathematics (especially Boolean algebra) and engineering, enabling the creation of everything from simple calculators to complex processors like CPUs.

1.1 Key Concepts

- **Digital vs. Analog:** Digital systems use discrete values (binary), while analog uses continuous signals. Digital is more reliable due to noise immunity.
- **History:** Originated in the 1930s with Claude Shannon's application of Boolean algebra to circuits. Evolved with transistors (1947), ICs (1958), and modern FPGAs/ASICs.
- **Importance:** Essential for computer architecture, embedded systems, VLSI design, and AI hardware. Understanding DLD helps in debugging hardware, optimizing performance, and innovating in IoT, robotics, etc.
- **Design Flow:** Starts with specifications, uses Boolean logic for simplification, simulates in tools, and implements in hardware.

DLD assumes basic knowledge of electricity (e.g., voltage levels for 0/1) but focuses on abstract logic.

2 Number Systems and Representations

Digital systems use binary (base-2) for simplicity in hardware.

2.1 Common Number Systems

- **Binary (Base-2):** Digits 0,1. E.g., $1011_2 = 1 \cdot 2^3 + 0 \cdot 2^2 + 1 \cdot 2^1 + 1 \cdot 2^0 = 11_{10}$.
- **Decimal (Base-10):** Everyday numbers.
- **Hexadecimal (Base-16):** Digits 0–9, A–F. Compact for binary (e.g., $1011_2 = B_{16}$). Used in memory addresses.
- **Octal (Base-8):** Digits 0–7. Less common now.

2.2 Conversions

- **Decimal to Binary:** Divide by 2, read remainders bottom-up. E.g., 13_{10} : $13/2 = 6$ r1, $6/2 = 3$ r0, $3/2 = 1$ r1, $1/2 = 0$ r1 $\rightarrow 1101_2$.
- **Binary to Decimal:** Sum powers of 2.
- **Binary to Hex:** Group 4 bits (e.g., $1011_2 = B_{16}$).
- **Fractional:** For decimals, multiply fractional part by base, take integers.

2.3 Signed Representations

- **Sign-Magnitude:** MSB as sign (0 positive, 1 negative). E.g., -5 in 4-bit: 1101.
- **1's Complement:** Invert bits for negative. E.g., -5 (0101 $\rightarrow$ 1010).
- **2's Complement:** 1's complement +1. E.g., -5 : $1010 + 1 = 1011$. Widely used for arithmetic (easy addition/subtraction).

2.4 Codes

- **BCD (Binary-Coded Decimal)**: 4 bits per decimal digit (e.g., $25_{10} = 0010\ 0101$ BCD).
- **Gray Code**: Adjacent codes differ by one bit (minimizes errors in rotary encoders).

3 Boolean Algebra

Boolean algebra formalizes logic operations, invented by George Boole (1854). In DLD, variables are 0/1.

3.1 Basic Operations

- **AND** ($\cdot$ or $\wedge$): 1 if all inputs 1.
- **OR** ($+$ or $\vee$): 1 if any input 1.
- **NOT** ($\neg$ or $\neg$): Inverts ($0 \rightarrow 1$, $1 \rightarrow 0$).
- **NAND**: NOT AND.
- **NOR**: NOT OR.
- **XOR** ($\oplus$): 1 if odd number of 1s.
- **XNOR**: NOT XOR.

3.2 Axioms and Theorems

Property	AND Form	OR Form
Identity	$A \cdot 1 = A$	$A + 0 = A$
Null	$A \cdot 0 = 0$	$A + 1 = 1$
Idempotent	$A \cdot A = A$	$A + A = A$
Complement	$A \cdot \bar{A} = 0$	$A + \bar{A} = 1$
Commutative	$A \cdot B = B \cdot A$	$A + B = B + A$
Associative	$(A \cdot B) \cdot C = A \cdot (B \cdot C)$	$(A + B) + C = A + (B + C)$
Distributive	$A \cdot (B + C) = (A \cdot B) + (A \cdot C)$	$A + (B \cdot C) = (A + B) \cdot (A + C)$
Absorption	$A \cdot (A + B) = A$	$A + (A \cdot B) = A$
De Morgan's	$\overline{A \cdot B} = \bar{A} + \bar{B}$	$\overline{A + B} = \bar{A} \cdot \bar{B}$

Table 1: Boolean Algebra Theorems

- **Simplification**: Use theorems to minimize gates (e.g., $A + \bar{A}B = A + B$).

3.3 Representations

- **Sum of Products (SOP)**: OR of AND terms (minterms).
- **Product of Sums (POS)**: AND of OR terms (maxterms).
- **Karnaugh Maps (K-Maps)**: Graphical simplification for 2–6 variables. Group 1s in powers of 2, read as SOP.

Example: $F(A, B) = \bar{A}B + AB = A$ (simplified from K-Map).

4 Logic Gates and Circuits

Gates are basic building blocks implemented in transistors (e.g., CMOS).

4.1 Basic Gates

Gate	Symbol	Truth Table (A B — Out)	Description
AND	•	00:0, 01:0, 10:0, 11:1	All high
OR	≥ 1	00:0, 01:1, 10:1, 11:1	Any high
NOT	1	0:1, 1:0	Invert
NAND		AND then NOT	Universal
NOR		OR then NOT	Universal
XOR	$= 1$	00:0, 01:1, 10:1, 11:0	Different
XNOR		XOR then NOT	Same

Table 2: Basic Logic Gates

- **Universal Gates:** NAND or NOR can implement any function.
- **Fan-In/Out:** Inputs per gate; outputs it can drive.
- **Propagation Delay:** Time for signal to pass through gate.

4.2 Circuit Design

- **Combinational:** Output depends only on current inputs (no memory).
- **Sequential:** Output depends on inputs and past states (has memory).

5 Combinational Logic Design

Circuits where outputs are functions of inputs only.

5.1 Key Components

- **Half Adder:** Adds two bits (Sum = $A \oplus B$, Carry = $A \cdot B$).
- **Full Adder:** Adds three bits (two inputs + carry-in). Cascaded for multi-bit adders.
- **Subtractors:** Similar, using 2's complement.
- **Multiplexers (MUX):** Selects one input from many (e.g., 2:1 MUX: $S = 0$ selects I_0 , $S = 1$ selects I_1).
- **Demultiplexers (DEMUX):** Routes input to one of many outputs.
- **Encoders/Decoders:** Encoder (binary to one-hot), Decoder (one-hot to binary).
- **Comparators:** Check equality or magnitude.
- **ALU (Arithmetic Logic Unit):** Performs add, subtract, AND, etc., based on control signals.

5.2 Design Process

1. Truth table from specs.
2. Boolean expression (SOP/POS).
3. Simplify (K-Map or Quine-McCluskey).
4. Implement with gates.

Example: 2-bit Adder uses two full adders.

6 Sequential Logic Design

Involves memory elements; outputs depend on history.

6.1 Flip-Flops (FF)

- Basic memory unit (1 bit).
- **SR Latch**: Set ($S = 1, R = 0: Q = 1$), Reset ($S = 0, R = 1: Q = 0$). Invalid $S = R = 1$.
- **D FF**: Q follows D on clock edge (edge-triggered).
- **JK FF**: Versatile ($J = 1, K = 0$: set; $J = 0, K = 1$: reset; $J = K = 1$: toggle).
- **T FF**: Toggles on $T = 1$.
- **Clock**: Synchronous signal for timing.

6.2 Counters

- **Ripple (Asynchronous)**: FFs cascade, each triggers next (simple but slow).
- **Synchronous**: All FFs use same clock (faster).
- **Up/Down**: Increment/decrement.
- **Modulo-N**: Counts 0 to $N - 1$ (e.g., decade counter: 0–9).

6.3 Registers

- Group of FFs (e.g., 8-bit register stores byte).
- **Shift Registers**: SISO, SIPO, PISO, PIPO (serial/parallel in/out).
- **Applications**: Data storage, shifting for multiplication.

6.4 Hazards and Timing

- **Glitches**: Temporary wrong outputs in combinational.
- **Setup/Hold Time**: Data stability around clock edge.
- **Metastability**: Rare FF instability.

7 Finite State Machines (FSM)

Models sequential behavior.

7.1 Types

- **Mealy:** Output depends on state and inputs.
- **Moore:** Output depends only on state (safer).

7.2 Design

1. State diagram (circles for states, arrows for transitions).
2. State table.
3. Assign binary codes to states.
4. Derive next-state and output logic.
5. Implement with FFs and combinational logic.

Example: Vending machine FSM (states: idle, coin inserted, dispense).

8 Memory and Programmable Logic

- **ROM (Read-Only Memory):** Fixed data (e.g., boot code). Types: PROM, EPROM, EEPROM.
- **RAM (Random Access Memory):** Read/write. SRAM (fast, volatile), DRAM (dense, needs refresh).
- **Cache:** Fast RAM near CPU.
- **Programmable Devices:**
 - **PLA/PAL:** Programmable arrays for custom logic.
 - **FPGA (Field-Programmable Gate Array):** Reconfigurable chips with LUTs (Look-Up Tables), FFs, and interconnects.
 - **CPLD:** Smaller than FPGA, non-volatile.

9 Hardware Description Languages (HDL)

Text-based design for simulation/synthesis.

- **Verilog:** C-like, procedural/behavioral.

```
1 module AND_Gate (input a, b, output y);  
2     assign y = a & b;  
3 endmodule
```

- **VHDL:** Ada-like, strongly typed.
- **Tools:** Xilinx Vivado, Intel Quartus for simulation, synthesis to FPGA/ASIC.
- **Levels:** Behavioral (high-level), RTL (Register Transfer Level), Gate-level.

10 Advanced Topics

- **Asynchronous Design:** No clock (faster but complex timing).
- **Power Optimization:** Low-power gates, clock gating.
- **Testing:** Fault models (stuck-at), DFT (Design for Testability).
- **VLSI:** Very Large Scale Integration (millions of gates).
- **Quantum Logic:** Emerging with qubits, but not classical DLD.
- **Challenges:** Moore's Law slowing; focus on parallelism, 3D ICs.

11 Applications

- **Computers:** CPUs use ALUs, registers, FSMs.
- **Embedded Systems:** Microcontrollers (e.g., Arduino).
- **Communication:** Error detection (parity, CRC).
- **Consumer Electronics:** Displays, sensors.
- **AI Hardware:** Accelerators like TPUs use custom logic.

12 Best Practices

- **Modularity:** Design hierarchical circuits.
- **Simulation:** Use tools like ModelSim to verify before hardware.
- **Minimization:** Reduce gates for cost/speed.
- **Safety:** Avoid race conditions in sequential.
- **Standards:** TTL/CMOS logic families for voltage levels.